

Accumulators in Tail Recursion

The disadvantage of tail recursion is that it requires rearranging your recursive function so that the last action is returning the recursive call's return value. This can make our recursive code even more unreadable. Indeed, the `factorial()` function in this chapter's *factorialTailCall.py* and *factorialTailCall.html* programs is a bit harder to comprehend than the version in Chapter 2's *factorialByRecursion.py* and *factorialByRecursion.html* programs.

In the case of our tail call `factorial()` function, a new parameter named `accum` follows the calculated product as recursive function calls are made. This is known as an *accumulator* parameter, and it keeps track of a partial result of a calculation that would otherwise have been stored in a local variable. Not all tail recursive functions use accumulators, but they act as a workaround for tail recursion's inability to use local variables after the final recursive call. Notice that in *factorialByRecursion.py*'s `factorial()` function, the recursive case was `return number * factorial(number - 1)`. The multiplication happens after the `factorial(number - 1)` recursive call. The `accum` accumulator takes the place of the `number` local variable.

Also notice that the base case for `factorial()` no longer returns `1`; rather, it returns the `accum` accumulator. By the time `factorial()` is called with `number == 1` and the base case is reached, `accum` stores the final result to return. Adjusting your code to use tail call optimization often involves changing the base case to return the accumulator's value.

You can think of the `factorial(5)` function call as transforming into the following `return`, as shown in [Figure 8-1](#).